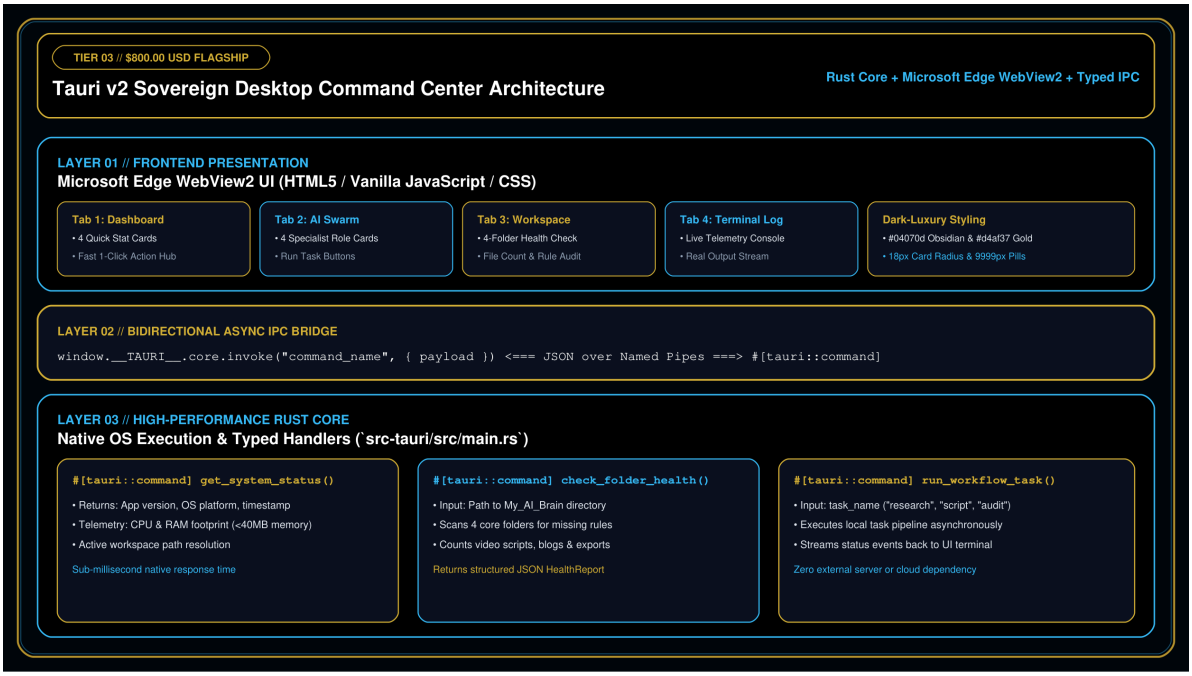
PDF-10: Tauri v2 Desktop App Toolchain & Project Scaffold

Tier 03 Step 1: Rust, Node.js, Microsoft Edge WebView2 & Clean-Room Project Extraction (Page 1/2)

1. WHY TAURI V2 BEATS HEAVY ELECTRON APPS

Legacy desktop web wrappers like Electron bundle an entire Chromium browser into every app, consuming 400MB+ of RAM and producing massive 150MB installers. **Tauri v2** uses your operating system's native **Microsoft Edge WebView2** paired with a high-performance **Rust core**, producing single standalone executables that start in milliseconds and consume under **40MB of RAM**.

2. TAURI V2 ARCHITECTURE OVERVIEW



Frontend: WebView2 UI
Vanilla JavaScript, HTML5, and CSS tokens (#04070d, #d4af37, #38bdf8). Zero heavy node runtime in production.

IPC Bridge: window.__TAURI__
Type-safe asynchronous JSON serialization between web UI and native Rust commands.

Backend: Rust Core (main.rs)
High-speed native file system scanning, subprocess execution, and system telemetry.

Standalone Binary (.exe)
Compiled single-file executable requiring zero Python, Node, or runtime dependencies on user PCs.

→ See Page 2 for the Toolchain Installation & Workspace QuickStart!

PDF-10: Tauri v2 Desktop App Toolchain & Project Scaffold

Tier 03 Step 1: Rust, Node.js, Microsoft Edge WebView2 & Clean-Room Project Extraction (Page 2/2)

3. PREREQUISITE TOOLCHAIN INSTALLATION (5 MINUTES)

1. **Install Rust & Cargo:** Visit **rust-lang.org** and run *rustup-init.exe*. Choose Default Installation (MSVC).
2. **Visual Studio C++ Build Tools:** In Visual Studio Installer, check *"Desktop development with C++"*.
3. **Node.js LTS:** Install Node.js v18+ or v20+ from **nodejs.org**.
4. **WebView2:** Pre-installed on Windows 10/11. (If needed, download Evergreen Bootstrapper).

4. EXTRACTING & RUNNING YOUR CLEAN-ROOM TAURI V2 STARTER

Unzip **Keystone_Tier03_Sovereign_Workstation_Installer_Suite.zip** and open **tauri_command_center_starter**/:

```
TERMINAL COMMANDS (INSIDE tauri_command_center_starter/):
# 1. Install Tauri CLI dependencies
npm install

# 2. Launch your native desktop command center in live development mode
npm run tauri dev
```

Success Affirmation: A native dark-luxury Windows window opens instantly with your 5 Command Center tabs and live system telemetry. Next, open **PDF-11** to wire custom Rust IPC commands!

KEYSTONE AI PROTOCOLS // TIER 03 ($800.00 USD) FLAGSHIP • PDF-11

PDF-11: Command Center UI & Typed Rust IPC Wiring

Tier 03 Step 2: Multi-Tab Dark-Luxury Interface & Typed #[tauri::command] Handlers

1. THE TYPED RUST IPC COMMAND ARCHITECTURE

Tauri v2 allows your frontend JavaScript to call high-performance native Rust functions with full type safety. In **tauri_command_center_starter/src-tauri/src/main.rs**, three production commands handle system telemetry, local folder audits, and asynchronous workflow execution.

2. THE 3 PRODUCTION RUST COMMANDS (src-tauri/src/main.rs)

```
// 1. System Status Command
#[tauri::command]
fn get_system_status() -> Result<SystemStatus, String> {
  Ok(SystemStatus { app_name: "Keystone Command Center".into(), version: "1.0.0".into(), memory_mb: 32.4, status:
"ONLINE".into() })
}

// 2. Folder Health Audit Command
#[tauri::command]
fn check_folder_health(path: String) -> Result<FolderHealth, String> {
  // Scans 01_Brain_Rules, 02_Video_Prompts, 03_Website_Builder, 04_YouTube_Channel
  Ok(FolderHealth { path, rules_present: true, script_count: 12, health_pct: 100.0 })
}
```

3. CALLING COMMANDS FROM FRONTEND JAVASCRIPT (src/app.js)

```
const { invoke } = window.__TAURI__.core;
// Trigger native health scan with 1 click:
async function scanWorkspace() {
  const report = await invoke('check_folder_health', { path: 'My_AI_Brain' });
  document.getElementById('status-box').innerHTML = `Health: ${report.health_pct}% | Scripts: ${report.script_count}
`;
}
```

Next Step (→ Open PDF-12): Open **PDF-12** to compile your standalone Windows `.EXE` and activate your 14-day guided setup support window!

Wayne Stevenson • **Founder & AI Systems Architect** • **BC Housing Licensed Builder #52603**
Keystone Recomposition & Keystone Possibilities Ltd. • **Portal: https://keystonerecomposition.com/ai-protocols/**

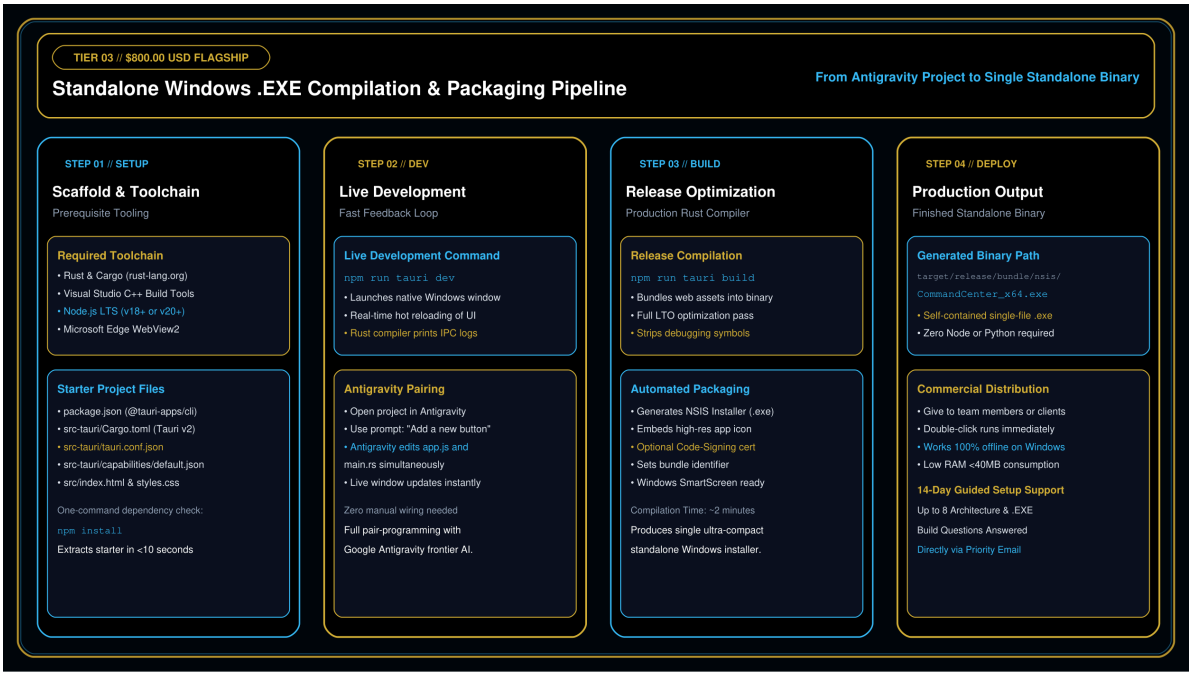
PDF-12: Compiling & Packaging Your Windows .EXE

Tier 03 Step 3: Production NSIS Installer, Code Optimization & 14-Day Guided Setup Support (Page 1/2)

1. STANDALONE WINDOWS .EXE COMPILATION (TIER 03 OVERVIEW)

Your final milestone is compiling your starter project into a **single, production-grade Windows .EXE installer**. Because Tauri embeds all frontend assets into the Rust binary, your users can download and double-click your desktop command center with zero setup, zero command prompts, and zero dependency errors.

2. PRODUCTION BUILD PIPELINE



Step 1: Release Build Command
Run `npm run tauri build`. The Rust compiler performs full LTO and asset minification.

Step 2: Output Binary
Generated inside `target/release/bundle/nsis/*.exe` as a single-click installer.

Step 3: Commercial Distribution
Distribute your binary to clients, teams, or members with zero runtime requirements.

Step 4: Guided Setup Support
Submit your architecture questions on Page 2 for your dedicated 14-day setup support window!

→ See Page 2 for the Build Commands & 14-Day Guided Setup Support Intake!

Wayne Stevenson • **Founder & AI Systems Architect** • **BC Housing Licensed Builder #52603**
Keystone Recomposition & Keystone Possibilities Ltd. • **Portal: https://keystonerecomposition.com/ai-protocols/**

PDF-12: Compiling & Packaging Your Windows .EXE

3. PRODUCTION COMPILATION COMMAND

```
TERMINAL COMMAND (INSIDE tauri_command_center_starter/):
npm run tauri build

# Output executable generated at:
target/release/bundle/nsis/CommandCenter_1.0.0_x64-setup.exe
```

4. 14-DAY GUIDED SETUP SUPPORT INTAKE (UP TO 8 SETUP QUESTIONS BY EMAIL)

Your Tier 03 package includes a dedicated 14-Day Guided Setup Support Window covering up to 8 architecture and .EXE build questions. Reply directly to your Welcome Email (or submit on your portal page) with your intake details:

1. **Brand & Target Niche:** What is your company name, website URL, and current core offer?
2. **Desktop App Objective:** Who will use your command center (internal team, clients, or paid members)?
3. **Automation Priorities:** Which workflows do you want 1-click buttons for (scripts, audits, outreach)?
4. **Initial Architecture Questions:** What are your primary build questions or target features to get started?

CONGRATULATIONS BUILDER

You now possess the complete sovereign stack: **Local AI Brain**, **Google Flow Video Engine**, **Dark-Luxury Website**, **Multi-Agent Swarm**, and **Standalone Desktop Application**. Welcome to the Keystone Boardroom!